

Datorlaboration 3

Josef Wilzén

September 2, 2026

732G12 Data Mining HT2024

Allmänt

Datorlaborationerna kräver att ni har R och Rstudio installerat.

- Kodmanual: [länk](#)
 - Utvärdering av klassificeringsproblem, se [här](#).
- **ISL**: An Introduction to Statistical Learning,
 - Boken: [länk](#)
 - R-kod till labbar: [länk](#)
 - Dataset: [länk](#) och [länk](#)
- **IDM**: Introduction to Data Mining
 - Kod till boken finns [här](#)
 - Sample chapters
- Dataset till vissa uppgifter finns [här](#).

Notera att ni inte behöver göra alla delar på alla uppgifter. Det viktiga är att ni får en förståelse för de olika principerna och modellerna som avhandlats. Dessa uppgifter ska inte lämnas in, utan är till för er övning.

Datauppdelning

För att motverka överanpassning bör ni dela upp data till träning-, validering-, (och testmängd). Detta kan göras med `createDataPartition()` från `caret`-paketet. Argument till den funktionen som är av vikt här är p som hur stor andel av observationerna som ska användas till träningsmängden. Ni kan också använda `subset()` för att göra detta också, men det blir svårare att tydligt ange de observationer som ska tilldelas till valideringsmängden. Denna uppdelning ska ske slumpmässigt. Notera att om en testmängd ska skapas måste uppdelningen ske en gång till från valideringsmängden eller träningsmängden.

Del 1: Trädmodeller

ISL

Gå igenom laborationerna 8.3.1 och 8.3.2 i **ISL**.

Begränsningar av beslutsträd

Läs in datamaterialet “issue.csv” som innehåller två förklarande variabler och en binär responsvariabel. Målet med denna övning är att ni ska skapa ett beslutsträd på detta data och undersöka kvaliteten av denna modell. Använd R-paketet `rpart`, se kodmanualen för exempel, se även IDM och denna R-fil, för att göra följande:

1. Läs in datamaterialet och se till att alla variabler följer rätt skala och dela sedan upp datamaterialet i 50 procent träning och 50 procent validering.
2. Skatta ett beslutsträd med där maxdjupet är satt till 5 och antal korsvalideringar är satt till 0 (i övrigt standardinställningar enligt kodmanualen).
3. Ta fram en plot över trädet med funktionen `rpart.plot` från paketet med samma namn. Se här för exempel på användning: [länk](#).
4. Utvärdera den skattade modellen genom att:
 - (a) Beräkna felkvoten av både tränings och valideringsmängden och tolka. Kolla tex här.
 - (b) Utforska modellen, hur komplex är den, hur många löv har valts ut och vilket utav de är den största?
 - (c) Hur djupt är det resulterande trädet?
5. Visualisera det ursprungliga datamaterialet och dess klasser. Hur ser beslutsgränsen ut?
6. Visualisera en liknande figur men nu endast på de predikterade värdena från valideringsmängden. (Ledning: För att prediktera nya utfall från en skattad modell används `predict(model, newdata = data, type = "class")`.) Verkar det som att beslutsgränsen som hittades i 5) också hittas med modellen? Vad för struktur har den nuvarande gränsen?
7. Skatta ett nytt beslutsträd där maxdjupet istället är satt till 10. Vad händer med den resulterande modellen och de predikterade värdena?
8. Sammanfatta era iakttagelser och svara på följande frågor:
 - (a) Varför kan det vara svårt att använda de två skattade modellerna?
 - (b) Varför kan det vara svårt att hitta en bra beslutsträdsmodell för detta data?

9. Skapa nya variabler i issue-materialet genom följande formler, $Z1 = X1 + X2$ och $Z2 = X1 - X2$. Visualisera datamaterialet i det nya koordinatsystemet och kommentera på beslutsgränsen.
10. Skatta ett beslutsträd likt den från uppgift 2) men använd istället det nya koordinatsystemet som förklarande variabler. Vad blir resultatet av denna modell, dess felkvoter i träning och valideringsmängden, antalet löv och djup? Varför skiljer sig resultatet här gentemot tidigare modeller?

Regression med beslutsträd

1. Återvänd till datasetet `lab2_data_1.csv` från laboration 2, länk.
2. Dela upp data i 70 % träning och 30 % validering.
3. Skatta beslutsträd på träningsdata. Testa några olika träd (minst tre olika) där ni ändrar maxdjup och/eller minsta antal observationer som behövs för en uppdelning. Ta fram MSE för träning och validering för de olika modellerna. Skapa plottar över träningsdata och anpassade värden från modellerna på träningsdata. Jämför och analysera. Hur påverkar de olika hyperparametrarna anpassningen? Den modell som ni är mest nöjd med: Ta fram en plot över det skattade trädet. Jämför det plottade trädet med er plot över anpassade värden på träningsdata.
4. Upprepa 3) men cost complexity pruning tillsammans med korsvalidering¹ för att välja modell. Hur komplext/djupt blev ert träd? Gör sedan prediktioner på valideringsdata och jämför validerings MSE med era resultat från 3).
5. Jämför med någon metod från kursvecka 2 (splines, loess etc)

¹Korsvalideringen görs på träningsdata.

Del 2: Ensemblemetoder

ISL

- Bootstrap:
 - Gå igenom laborationen 5.3.4
- Bagging and Random Forests, Boosting
 - Gå igenom laborationerna 8.3.3 och 8.3.4
 - 8.4 Exercises Conceptual: 2, 5

Del 3: XGBoost

1. Testa koden här: [länk](#). Undersök datasetet, modellens inställningar och studera resultatet från modellanpassningen. Anser ni att det är en bra anpassning eller ej?
2. Klassificering: Korsvalidering och XGBoost. Gå igenom skriptet här: [länk](#)
 - Först skattas en “första gissning”-modell
 - Sen skattas sex olika modeller med olika hyperparametrar och korsvalidering används för att välja den bästa kombinationen av hyperparametrar.
 - Studera resultatet från de olika modellerna. Hur blir anpassningen?
3. Klassificering: Utgå från koden i steget innan, men testa nu sex andra hyperparameterkombinationer som ni själva bestämmer. Utvärdera och analysera resultatet. Blir anpassningen bättre eller sämre jämfört med de modeller som skattades i föregående steg?
4. Regression: Korsvalidering och XGBoost. Gå igenom skriptet här: [länk](#)
 - Först skattas en “första gissning”-modell
 - Sen skattas sex olika modeller med olika hyperparametrar och korsvalidering används för att välja den bästa kombinationen av hyperparametrar.
 - Studera resultatet från de olika modellerna. Hur blir anpassningen?
5. Regression: Utgå från koden i steget innan, men testa nu sex andra hyperparameterkombinationer som ni själva bestämmer. Utvärdera och analysera resultatet. Blir anpassningen bättre eller sämre jämfört med de modeller som skattades i föregående steg?

Del 4: Identifiering av spam-mail

Filen “spambase.csv” innehåller information om frekvensen av ett antal ord, symboler osv. från 4601 olika mail. Dessa har också klassificerats som antingen spam (spam = 1) eller vanliga mail (spam = 0). Er uppgift är nu att skatta olika modeller som kan användas som någon form av spamfilter för nya mail som anländer till inkorgen. Mer detaljerad information om dessa attribut är angivna i bilagan nedan. Då våra förklarande är tungsvansade kan en log-transformation fungera bra ($\log(x + 0.1)$) där vi lägger på 0.1 för att undvika $\log(0)$ problem.

1. Läs in materialet och se över variabelernas skalor enligt dess beskrivning. Hur är klassbalansen i responsvariabeln? Hur många observationer och variabler finns det?
2. Skapa följande uppdelning av data: 50 % träning, 30 % validering och 20 % test. Se till att det är ungefär samma klassbalans i alla tre delarna.
3. Skatta följande modeller:
 - (a) Linjär regression med LASSO, bestäm λ med korsvalidering på träningsdata.
 - (b) Skatta ett betlutsträd. Använd cost complexity pruning tillsammans med korsvalidering på träningsdata för att välja modell.
 - (c) Skatta Random forest-modell med $mtry=\sqrt{p}$, där p är antalet förklarande variabler och sätt antal träd till 300.
 - (d) Skatta en XGBoost modell. Undersök några hyperparameter kombinationer som ni själva väljer. Välj bästa kombination hyperparameter genom att undersöka träffsäkerhet på valideringsdatan eller använd korsvalidering på träningsdatan.
4. Ta fram träffsäkerhet för alla modellerna på träningsdata och valideringsdata. Vilken modell verkar vara bäst? Undersök även sensitivitet och specificitet. Analysera dessa resultat och välj sedan en modell som er “bästa” modell.
5. Ta er bästa modell från föregående steg och skatta om den med hjälp av träningsdata + valideringsdata (=80 % av all data). Undersök träffsäkerhet, sensitivitet och specificitet på denna modellskattning på den nya större träningsdatan.
6. Ta modellen från föregående steg beräkna träffsäkerhet, sensitivitet och specificitet på testdata. Analysera resultatet. Anser ni att den bästa modellen skulle vara ett bra spamfilter eller ej?

Del 5: Mer övningsuppgifter

Tillämpning

- **ISL 5.4 Exercises Applied:** 9
- **ISL 8.4 Exercises Applied:** 7, 8, 9, 10, 11

Teori

1. Beskriv för- och nackdelarna med för- och efterbeskränning och svara vilken av metoderna som vanligtvis föredras.
2. Vad är en beslutsgräns och vilken form följer den i beslutsträd?
3. Vilken skala är bäst anpassad för klassificeringsproblem generellt, nominal eller ordinal? Varför?
4. Hur hanteras ordinala förklarande variabler i beslutsträd?
5. Klassificeringsträd: varför är det inte så bra att använda felkvoten (misclassification rate) när nya beslutsgränser ska skapas?
6. Regressionsträd: Vilken kostnadsfunktion brukar används för att utvärdera beslutsgränser?
7. Utan regularisering: vad brukar vara ett problem för trädmodeller, bias eller varians?
8. Utgå från ett kontinuerligt y och en kontinuerlig förklarande variabel x . Rita upp en funktion (på papper) $y = f(x)$ som är lätt att anpassa med ett beslutsträd, men som är problematisk för linjär regression.
9. **ISL 5.4 Exercises Conceptual:** 4
10. **ISL 8.4 Exercises Conceptual:** 1, 3

Bilaga

Attribute Information for spambase: The last column of “spambase.csv” denotes whether the e-mail was considered spam (1) or not (0), i.e. unsolicited commercial e-mail. Most of the attributes indicate whether a particular word or character was frequently occurring in the e-mail. The run-length attributes (55-57) measure the length of sequences of consecutive capital letters. For the statistical measures of each attribute, see the end of this file.

Here are the definitions of the attributes: 48 continuous real [0,100] attributes of type word_freq_WORD = percentage of words in the e-mail that match WORD, i.e. $100 * (\text{number of times the WORD appears in the e-mail}) / \text{total number of words in e-mail}$. A “word” in this case is any string of alphanumeric characters bounded by non-alphanumeric characters or end-of-string.

- 6 continuous real [0,100] attributes of type char_freq_CHAR = percentage of characters in the e-mail that match CHAR, i.e. $100 * (\text{number of CHAR occurrences}) / \text{total characters in e-mail}$
- 1 continuous real [1, . . .] attribute of type capital_run_length_average = average length of uninterrupted sequences of capital letters
- 1 continuous integer [1, . . .] attribute of type capital_run_length_longest = length of longest uninterrupted sequence of capital letters
1 continuous integer [1, . . .] attribute of type capital_run_length_total = sum of length of uninterrupted sequences of capital letters = total number of capital letters in the e-mail
- 1 nominal {0,1} class attribute of type spam = denotes whether the e-mail was considered spam (1) or not (0), i.e. unsolicited commercial e-mail.